

Group 5: Report

DSAA CA2

2024

Presented by:

DAAA/2B/01

Teng Tze Hui (P2214209)

Ng Jace Xin (P2214593)

1.0 USER GUIDELINES

1.1 Description

This application facilitates the evaluation of fully parenthesized mathematical expressions containing variables. Users can seamlessly add assignments through direct input or by reading from an input file. A distinctive feature enables the evaluation and printing of parse trees for stored variables. Additionally, users can modify assignments individually, ensuring an interactive experience. The application ensures a streamlined presentation by sorting and printing values alphabetically, enhancing accessibility and ease of analysis.

1.2 Loading the Program

Load the program by opening Anaconda Prompt and going into the project folder using the `cd` command. Afterwards, at the command prompt type in `python main.py` to access the program.

Once you press `enter`, you will be prompted with the menu, showing options 1 to 10. From here, you will be able to access the various functions available.

```
(base) C:\Users\tengt>cd C:\DSAA-CA2
(base) C:\DSAA-CA2>python main.py

*****
* ST1507 DSAA: Evaluating & Sorting Assignment Statements *
*****
* -----*
* *
* - Done by: Teng Tze Hui (P2214209) & Ng Jace Xin (P2214593) *
* - Class DAAA/2B/01 *
* *
*****

Press enter key, to continue...

Please select your choice: (1,2,3,4,5,6,7,8)
    1. Add/Modify assignment statement
    2. Display current assignment statements
    3. Evaluate a single variable
    4. Read assignment statements from file
    5. Sort assignment statements
    6. Visualizing assignment statements with graphs (Jace)
    7. Turtle Equation Drawing (Jace)
    8. Sorting Game (Tzehui)
    9. Graph Plotting Features (Tzehui)
    10. Exit
Enter Choice: |
```

1.3 Program Usage

Add/Modify assignment statement

- Enter choice `1` and you will be prompted to give an assignment statement. Key in the assignment statement in the given format in the example.

```
Enter Choice: 1
Enter the assignment statement you want to add/modify:
For example, a=(1+2)
a=(1+2)
```

- The program includes validation to ensure that assignment statements are syntactically complete.
- However, it does not perform checks to ensure that the given statements are valid mathematical expressions

```
Enter Choice: 1
Enter the assignment statement you want to add/modify:
For example, a=(1+2)

Invalid input. Please enter a valid statement.
Enter the assignment statement you want to add/modify:
For example, a=(1+2)
a=
Invalid incomplete statement. Please enter a complete assignment.
Enter the assignment statement you want to add/modify:
For example, a=(1+2)
a=(
Invalid statement. Please include at least one of the operators: +, -, *, /, **
Enter the assignment statement you want to add/modify:
For example, a=(1+2)
a=(1+
Invalid incomplete statement. Please enter a complete assignment.
Enter the assignment statement you want to add/modify:
For example, a=(1+2)
a=(1+@)
```

Display current assignment statements

- Enter choice **2**, and you will be presented with the current assignment statements along with their corresponding evaluated results.
- If you **skip option 1**, and **jump straight to option 2**, no assignment statements will be printed except the outline.
- If previously, you did not enter valid mathematical expressions in option 1. The program will raise the error and put the evaluation as none.
- Do note that the expression can handle float, variables, and integers.

```
CURRENT ASSIGNMENTS:
*****
A = (1/4) => 0.25
B = (3**2) => 9
```

```
CURRENT ASSIGNMENTS:
*****
```

```
Error during tokenization: Invalid character: @
```

```
CURRENT ASSIGNMENTS:
*****
a = (1+@) => None
```

Evaluate a single variable

- Enter choice **3**, and you will be prompted to ask for the variable you would like to see the parse tree.
- If you **skip option 1** and **jump straight to option 3 directly**, the application will state the variable is not found in the storage.
- Afterward, you will be returned to the menu. This also works if you did not key in any variables into the prompt.

```
Please enter the variable you want to evaluate:
Apple

Expression Tree:
..5
.*
..4
+
.2
Value for variable 'Apple' is 22
```

```
Please enter the variable you want to evaluate:
a

Expression Tree:
Variable 'a' not found in statement storage.
```

Read assignment statements from file

- Enter choice **4**, and you will be prompted to input a text file. It will validate and ensure that the text file exists before reading the assignment statements.
- Afterwards, the results will be printed out and sorted in alphabetical order.

```
Enter Choice: 4
Please enter input file:
Invalid input. Please enter a valid file path.
Please enter input file: 0
Invalid input. Please enter a valid file path.
Please enter input file: abc
Invalid input. Please enter a valid file path.
Please enter input file: fruits.txt

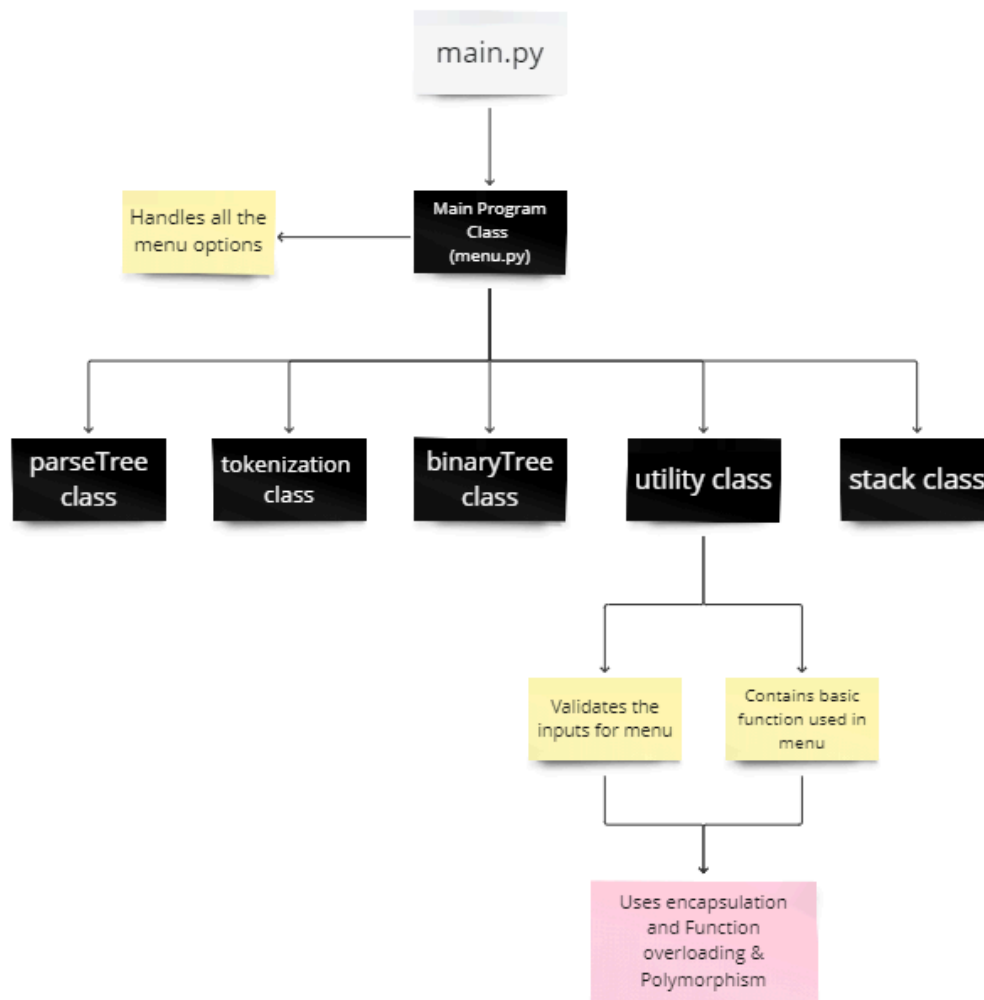
CURRENT ASSIGNMENTS:
*****
Apple = (3*5) => 15
Banana = (2*(65.5+2.25)) => 135.5
Blueberry = (Pear+0) => 16
Cherry = (Lemon/2) => None
Coconut = (1+0) => 1
Durian = (Apple*Pear) => 240
Lemon = (Pear+Tangerine) => None
Mango = ((Apple+(Durian+(Pear*(Blueberry*(Coconut/Strawberry)))))/2) => 135.5
Pear = (Apple+1) => 16
Starfruit = (2**4) => 16
Strawberry = (Pear*1) => 16
```

Sort assignment statements

- Enter choice **5**, and you will be prompted to input an output file. It will validate and ensure that the input given is not empty.
- Afterwards, the results will be written into the output file and a success message will be printed
- Do note that the option will not work if you **jump straight to option 5 without doing option 4**.

```
Enter Choice: 5
Please enter output file:
Error writing to the output file: [Errno 2] No such file or directory: ''
Please enter output file: fruits_sorted2.txt
Output file 'fruits_sorted2.txt' created successfully.
```

2.0 OBJECT ORIENTED PROGRAMMING (OOP) OVERVIEW



2.1 General OOP Practices

We practice encapsulation, function overloading and polymorphism. Making use of private variables like `__validateFilePath`.

Private Variables:

- a. The use of `__validateFilePath` is to ensure that a given file path is valid before further processing
- b. The use of `__validateOutPath` is to ensure that the file name that the user wants to write to does not exist.

Polymorphism:

- a. `enterChoice(prompt)` expects the user to enter an integer. If a non-integer is entered, a `ValueError` is raised. This function demonstrates polymorphism as it works with different data types.
- b. `getFile(filePathPrompt)` takes a file path as input and ensures its validity. This function can be seen as polymorphic as it can handle various file paths.
- c. `processAssignmentStatements(content, statement_storage)` processes assignment statements from the input content. The function can work with different content and store assignments in the `statement_storage` dictionary, showcasing polymorphism.

2.2 Main Program Class

The main part of our program is handled by the `Menu` class. When we create an instance of the `Menu` class in the `main.py` file, it starts the program and handles different features for user interaction.

2.3 Parse Tree Class

The `Parse Tree` class is a fundamental component of our application. It plays a crucial role in the construction and evaluation of parse trees for mathematical expressions. This class encapsulates the logic to navigate through expressions, build hierarchical structures and compute results using the PEMDAS rule. Leveraging binary tree logic, the class efficiently represents the inherent structure of mathematical expressions, aiding in the seamless construction of parse trees.

2.4 Tokenization Class

The tokenization process in our application serves as the initial phase in transforming raw mathematical expressions into a format suitable for parsing and evaluation. This process involves breaking down expressions into individual tokens, which can be operators, variables, or numerical values. The `Tokenization` class is responsible for implementing this essential functionality.

2.5 Binary Tree Class

The BinaryTree class is a foundational element in our application, serving as a core data structure for constructing and manipulating binary trees. This class encapsulates the logic required to manage nodes within the tree, including methods for setting and retrieving keys, inserting nodes on the left and right, and printing the tree in both preorder and inorder traversals.

2.6 Utility Class

The Utility class serves as a collection of helpful functions and utilities that are commonly used across the application. It streamlines user input, file processing, and various validation tasks, contributing to a more modular and readable codebase.

2.7 Stack Class

The Stack class is a fundamental data structure used in various parts of the application. It provides a last-in, first-out (LIFO) mechanism, allowing efficient handling of data sequentially. Below is an overview of the key functionalities and characteristics of the Stack class.

3.0 DATA STRUCTURES

Data Structure	Characteristics	Use Cases	Insertion/ Deletion O
List	An ordered data structure that stores a collection of items, where each item can be of any data type	Managing collections of items Maintaining a sequence of elements Dynamic resizing for efficient insertions/deletions Random access to elements by index Stack and Queue implementations	Access: $O(n)$ - Direct access to elements by index. Append: $O(n)$ - Adding an element to the end of the list. Insert/Delete: $O(n)$ - Inserting or deleting elements in the middle requires shifting elements. Search: $O(n)$ - Linear search through the list. Slice: $O(n)$ - Extracting a subsequence of 'n' elements.
Binary Search Tree (BST)	A binary search tree is used for parse tree representation	Representing mathematical expressions in a tree format	$O(\log n)$ - On average for balanced trees

Stack	A linear data structure that follows a particular order in which the operations are performed. Last In First Out basis (LIFO)	Expression evaluation, backtracking, parsing	$O(n)$ - Inserting or deleting elements from the top
ParseTree	A tree structure representing parsed mathematical expressions	Evaluating mathematical expressions	$O(n)$ - Building the parse tree from an expression
Tuple	An ordered, immutable collection of elements	Grouping related data, returning multiple values	$O(n)$ for access

4.0 GROUP REFLECTION

4.1 Challenges faced

One of the obstacles we encountered during the development of our program was the constraint of time. There was limited time available for refining the program, enhancing user interfaces, improving code efficiency, and implementing additional features that we envisioned.

Another challenge we faced was collaborating on our work across different devices. We addressed this issue by uploading our files to Github. With a bit of coordination and communication, this approach proved effective in overcoming the challenge of working on separate devices.

4.2 Takeaways

Jace: I learned how the different algorithms work, such as the Binary Tree and Parse Tree, and implemented them into the program. I also learned how to manage my time and communicate effectively in a team to ensure that we are on the same page and are making progress.

Tze Hui: I learned to integrate Object-Oriented Programming (OOP), sorting, searching, trees, and algorithms into a program. Additionally, I acquired the skill of collaborating effectively with my teammate online, allowing us to better improvise on the program's features together.

4.3 Roles and contributions

Jace: Implemented Tokenization class, options 2 and 5. Improved on ParseTree and Stack classes. Improved on option 1 and checked for errors.

Tze Hui: Implemented Utility, Menu class, options 1, 3, 4. Improved on ParseTree evaluate function and checked for errors.

APPENDIX: SOURCE CODE

Source Code Files	Done by
binaryTree.py	Tze Hui
main.py	Jace, Tze Hui
menu.py	Jace, Tze Hui
parseTree.py	Jace, Tze Hui
stack.py	Jace, Tze Hui
tokenization.py	Jace
utility.py	Jace, Tze Hui
drawTree.py (individual)	Jace
turtleGraph.py (individual)	Jace
graph.py (individual)	Tze Hui
sort.py (individual)	Tze Hui

binaryTree.py
<pre> # Name: Teng Tze Hui & Ng Jace Xin # Student ID: 2214209 & 2214593 # Class: DAA/2B/01 class BinaryTree: def __init__(self, key, leftTree = None, rightTree = None): self.key = key self.leftTree = leftTree self.rightTree = rightTree def setKey(self, key): self.key = key def getKey(self): return self.key def getLeftTree(self): </pre>


```
        return self.leftTree

def getRightTree(self):
    return self.rightTree

def insertLeft(self, key):
    if self.leftTree == None:
        self.leftTree = BinaryTree(key)
    else:
        t =BinaryTree(key)
        self.leftTree , t.leftTree = t, self.leftTree

def insertRight(self, key):
    if self.rightTree == None:
        self.rightTree = BinaryTree(key)
    else:
        t =BinaryTree(key)
        self.rightTree , t.rightTree = t, self.rightTree

def printPreorder(self, level):
    print( str(level*'.') + str(self.key))
    if self.leftTree != None:
        self.leftTree.printPreorder(level+1)
    if self.rightTree != None:
        self.rightTree.printPreorder(level+1)

def printInorder(self, level):
    if self.rightTree != None:
        self.rightTree.printInorder(level+1)
    print( str(level*'.') + str(self.key))
    if self.leftTree != None:
        self.leftTree.printInorder(level+1)
```

main.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAA/2B/01
```

```
# Imports
from menu import Menu

# Main Program
if __name__ == "__main__":
    app = Menu()
```

menu.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

# Imports
from utility import Utility
from parseTree import parseTree
from sort import Sort
from graph import GraphUtility
from drawTree import DrawTree
from turtleGraph import EquationGraph

class Menu:

    # Intialise self to run the few programs when called
    def __init__(self):
        self.statement_storage = {}
        self.variable_values = {}
        self.printOutline()
        self.selection()
        self.sorted_statements = None

    # Print the outline
    def printOutline(self):
        print('\n' + '*' * 65)
        print('* ST1507 DSAA: Evaluating & Sorting Assignment
Statements' + '\t' + '*')
        print('*' + '-' * 63 + '*')
```

```
print('*' + '\t'*8 + '*')
print('* - Done by: Teng Tze Hui (P2214209) & Ng Jace Xin
(P2214593)' + '\t' + '*')
print('* - Class DAAA/2B/01' + '\t' * 6 + '*')
print('*' + '\t' * 8 + '*')
print('*' * 65 + '\n' * 2)

# Show the menu
def showMenu(enter):
    print('\nPlease select your choice: (1,2,3,4,5,6,7,8)')
    print('\t 1. Add/Modify assignment statement')
    print('\t 2. Display current assignment statements')
    print('\t 3. Evaluate a single variable')
    print('\t 4. Read assignment statements from file')
    print('\t 5. Sort assignment statements')
    print('\t 6. Visualing assignment statements with graphs
(Jace)')
    print('\t 7. Turtle Equation Drawing (Jace)')
    print('\t 8. Sorting Game (Tzehui)')
    print('\t 9. Graph Plotting Features (Tzehui)')
    print('\t 10. Exit')

# Functions for each option the user choose
def selection(self):
    while True:
        Utility.userInput()
        self.showMenu()

        # print the enter choice
        choice = Utility.enterChoice('Enter Choice: ')

        if choice == 1:
            self.option1()
        elif choice == 2:
            self.option2()
        elif choice == 3:
            self.option3()
        elif choice == 4:
```

```
        self.option4()
    elif choice == 5:
        self.option5()
    elif choice == 6:
        self.option6()
    elif choice == 7:
        self.option7()
    elif choice == 8:
        self.option8()
    elif choice == 9:
        self.option9()
    elif choice == 10:
        self.option10()
    return
else:
    print("Invalid operation. Please try again and enter the
correct choices.")

    print()

# Option 1
def option1(self):
    validated_input = Utility.validateVarName()

    # If the validation is successful, update the class attributes
    if validated_input:
        var, exp = validated_input

        # Modifying existing assignment statement
        if var in self.statement_storage:
            self.statement_storage[var] = exp
        else:
            # Store the variables and expression in a dictionary
            self.statement_storage[var] = exp

# Option 2
def option2(self):
    # Sort the assignment statements alphabetically
```

```
        self.sorted_statements = sorted(self.statement_storage.items(),
key=lambda x: x[0])

        while True:
            # Store the current variable values to check for changes
later
            current_values = self.variable_values.copy()

            # Iterate through sorted statements
            for var, exp in self.sorted_statements:
                if var not in self.variable_values:

                    tree = parseTree(exp, self.variable_values) #
evaluate using the parseTree class
                    new_value = tree.result

                    if new_value is not None:
                        self.variable_values[var] = new_value

                    # Check if the expression has changed
                    elif exp != self.variable_values.get(var):
                        # Parse the modified expression using the current
variable values
                        tree = parseTree(exp, self.variable_values)

                        # Get the result of the modified expression
                        new_value = tree.result

                        # If the result is not None, update the variable
value and expression
                        if new_value is not None:
                            self.variable_values[var] = new_value

                    # Check for changes in variable values
                    if current_values == self.variable_values:
                        break

            # Print the format
```

```
print('\nCURRENT ASSIGNMENTS:')
print('*' * 20)

# Print the final sorted statements
for var, exp in self.sorted_statements:
    evaluated_exp = self.variable_values.get(var, None)
    correct_exp = exp.replace(" ", "")
    if evaluated_exp is not None:
        print(f'{var} = {correct_exp} => {evaluated_exp}')
    else:
        print(f'{var} = {correct_exp} => None')

# Option 3 -- format and evaluate all the variables
def option3(self):
    # Get the user input
    var = Utility.userInput('Please enter the variable you want to
evaluate:\n')
    print('\nExpression Tree:')

    if var in self.statement_storage:
        exp = self.statement_storage[var]

        # build the parse tree for evaluation + parse tree
        parsedTree = parseTree(exp, self.variable_values)

        # Store the result in self.variable_values
        self.variable_values[var] = parsedTree.result

        # evaluate the statement
        evaluated_exp = parsedTree.result

        # print the parse tree out
        parsedTree.tree.printInorder(0)
        print(f'Value for variable \'{var}\'' is {evaluated_exp}')
    else:
        print(f'Variable \'{var}\'' not found in statement storage.')

# Option 4
```

```
def option4(self):
    # Get and Read the file
    filePath = Utility.getFile('Please enter input file: ')
    readFile = Utility.readFile(filePath)

    # Process assignment statements
    Utility.processAssignmentStatements(readFile,
self.statement_storage)

    # Sort the assignment statements alphabetically
    self.sorted_statements = sorted(self.statement_storage.items(),
key=lambda x: x[0])

    while True:
        # Store the current variable values to check for changes
later
        current_values = self.variable_values.copy()

        # Iterate through sorted statements
        for var, exp in self.sorted_statements:
            if var not in self.variable_values:
                tree = parseTree(exp, self.variable_values)
                new_value = tree.result

                if new_value is not None:
                    self.variable_values[var] = new_value

            # Check if the expression has changed
            elif exp != self.variable_values.get(var):
                # Parse the modified expression using the current
variable values
                tree = parseTree(exp, self.variable_values)

                # Get the result of the modified expression
                new_value = tree.result

                # If the result is not None, update the variable
value and expression
```

```
        if new_value is not None:
            self.variable_values[var] = new_value

    # Check for changes in variable values
    if current_values == self.variable_values:
        break

    # Print the format
    print('\nCURRENT ASSIGNMENTS:')
    print('*' * 20)

    # Print the final sorted statements
    for var, exp in self.sorted_statements:
        evaluated_exp = self.variable_values.get(var, None)
        correct_exp = exp.replace(" ", "")
        if evaluated_exp is not None:
            print(f'{var} = {correct_exp} => {evaluated_exp}')
        else:
            print(f'{var} = {correct_exp} => None')

def option5(self):
    # Check if sorted_statements is available
    try:
        if self.sorted_statements is None:
            print("Sorted statements are not available. Please run
option 4 first.")
            return
    except AttributeError:
        print("Sorted statements are not available. Please run
option 4 first.")
        return

    # Pass self.variable_values to Utility.validateOutFile
    outputFile = Utility.getOutputFile("Please enter output file",
self.sorted_statements, self.variable_values)
    print(f"Output file '{outputFile}' created successfully.")

    # Additional Features 1 (Jace)!
```



```
# let users visualise the graph
def option6(self):

    print("Hello there! \nThis tree is deisgned to help you
understand the flow of dependencies between the variables you have
input!\n")
    Utility.userInput()

    print("\nIf the graph is empty, it means that there are no
relationships between the current variables stored..\n")
    Utility.userInput()

    if not self.statement_storage:
        print("\nOh no! It seems like your tree is empty, please
select options 1 or 4 to input variables!")
        return

    tree = DrawTree(self.statement_storage)

    tree_data = tree.build_tree_data(self.statement_storage)
    tree.draw_tree(tree_data)

    user_choice = ""
    while user_choice not in ["yes", "no", "y", "n"]:
        user_choice = Utility.userInputNotEmpty('\nDo you want to
display the dependency matrix information? (yes/no): ').lower()

        if user_choice == "yes" or user_choice == "y":
            print(self.statement_storage)
            tree.visualize_dependency_info()
        elif user_choice == 'no' or user_choice == 'n':
            return
        else:
            print('Please enter "yes" or "no"!')

# Additional Features 2 (Jace)!
def option7(self):
```

```
        equation_graph = EquationGraph() # Create an instance of
EquationGraph
        equation_graph.input_equations() # Call the input_equations()
method
        color, linestyle = equation_graph.customize_graph()
        print("Your graph will be printed shortly.. Click on the graph
to get the axis points!")
        equation_graph.draw_graph(color, linestyle)

# Additional Features 3 (Tzehui)!
def option8(self):
    # Introduction to the game
    print("\nWelcome to the Sorting Game!")
    print("This game is designed to help you understand various
sorting algorithms and their performance. You can interactively observe
how different algorithms work on a given set of data.\n")
    Utility.userInput()

    # Ask the user to choose an option
    print('\nChoose an option:\n1. Input your own data\n2. Generate
random data\n')
    option = Utility.enterChoice('Enter a choice: ')

    if option == 1:
        # Get the data from the user
        data = Utility.userInputNotEmpty('\nEnter a list of elements
(split by \',\'): ')
        elements = data.split(',')
        original_element = elements.copy()

    elif option == 2:
        # Generate random data based on user input (strings)
        size = Utility.enterChoice("\nEnter the size of the random
data: ")
        str_length = Utility.enterChoice("Enter the length of each
random string: ")
        elements = Utility.generateRandomData(size, str_length)
        original_element = elements.copy()
```

```
        else:
            print('It is not one of the valid options! Returning back to
main menu...')
            return

        # Get the user input for the sorting algorithm
        print('\nChoose a sorting algorithm:\n1. Bubble Sort\n2. Merge
Sort')

        choice = Utility.enterChoice('Enter a choice: ')

        # Create an instance of the Sort class
        sort = Sort()

        # Perform sorting based on the choice
        if choice == 1:
            sort.bubbleSort(elements)
        elif choice == 2:
            sort.mergeSort(elements)
        else:
            print('It is not one of the valid choices! Returning back to
main menu...')
            return

        # Display the results
        print(f'\nOriginal List: {original_element}')
        print(f'Sorted List: {elements}')
        print(f'Number of steps: {sort.steps}')

    # Additional Features 4 (Tzehui)!
    def option9(self):
        print('\nWelcome to the Graph Features')
        print("This feature is to help you to explore graph features
interactively. Visualize and analyze graphs with different layouts and
algorithms.\n")
        Utility.userInput()

        # Get user input for the graph data (edge list format)
```

```
    edgelist = Utility.userInputNotEmpty('\nEnter the edge list
(format: node1,node2,weight; e.g., A,B,2 B,C,1 A,C,3 D,A,4): ')

    # Convert input to a list of tuples (edge, weight)
    edges = [tuple(map(str.strip, edge.split(','))) for edge in
edgelist.split() if edge]

    # Create a graph
    g = GraphUtility.createGraph(edges)

    if g is None:
        return # Exit the option if creating the graph fails

    # Choose whether to find the shortest path
    findShortestPath = Utility.userInputNotEmpty('\nDo you want to
find the shortest path? (yes/no): ').lower()

    if findShortestPath == 'yes' or findShortestPath == 'y':
        # Get user input for shortest path
        start_node = Utility.userInputNotEmpty('Enter the start
node: ')
        end_node = Utility.userInputNotEmpty('Enter the end node: ')

        GraphUtility.findShortestDistance(g, start_node, end_node)

    elif findShortestPath == 'no' or findShortestPath == 'n':
        pass
    else:
        print('It is not one of the valid choices! Returning back to
main menu...')

    # Choose whether to visualize the graph
    visualize_graph = Utility.userInputNotEmpty('\nDo you want to
visualize the graph? (yes/no): ').lower()

    if visualize_graph == 'yes' or visualize_graph == 'y':
        # Choose plotting options
        print('\nChoose a graph layout:')
```

```
print('1. Normal Graph')
print('2. Minimum Spanning Tree')
print('3. Google Page Ranking Algorithm')

choice = Utility.enterChoice('Enter a choice: ')

GraphUtility.visualizeGraph(g, edges, choice)

elif visualize_graph == 'no' or visualize_graph == 'n':
    pass
else:
    print('It is not one of the valid choices! Returning back to
main menu...')

# Option 10 (EXIT!!!)
def option10(self):
    print('\nBye, thanks for using ST1507 DSAA: Evaluating & Sorting
Assignment Statements')
```

parseTree.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

# Imports
from stack import Stack
from binaryTree import BinaryTree
from tokenization import Tokenization
from decimal import Decimal

class parseTree():
    def __init__(self, exp, var_storage_statement=None):
        self.exp = exp
        self.tree = self.buildParseTree(exp)
        self.result = self.evaluate(self.tree, var_storage_statement)

# This tokenisation only look at the spaces in between
```

```
def buildParseTree(self, exp):
    # take out the tokensiation
    tokens = Tokenization.tokenize(exp) # look at the spaces
    stack = Stack()
    tree = BinaryTree('?')
    stack.push(tree)

    currentTree = tree

    for t in tokens:
        # RULE 1: If token is '(' add a new node as left child
        # and descend into that node
        if t == '(':
            currentTree.insertLeft('?')
            stack.push(currentTree)
            currentTree = currentTree.getLeftTree()

        # RULE 2: If token is operator set key of current node
        # to that operator and add a new node as right child
        # and descend into that node
        elif t in ['+', '-', '*', '/', '**']:
            currentTree.setKey(t)
            currentTree.insertRight('?')
            stack.push(currentTree)
            currentTree = currentTree.getRightTree()

        # RULE 3: work with the other variables in the statement (if
any)
        elif t.isalpha():
            currentTree.setKey(str(t))
            parent = stack.pop()
            currentTree = parent

        # RULE 4: If token is number, set key of the current node #
change this so that it accepts a variable as an input oso !!
        # to that number and return to parent
        elif t not in ['+', '-', '*', '/', '**', ')']:
            try:
```

```

        currentTree.setKey(int(t)) # Try to convert to
integer
        except ValueError:
            currentTree.setKey(Decimal(t)) # If conversion
fails, use float
        parent = stack.pop()
        currentTree = parent

        # RULE 5: If token is ')' go to parent of current node
    elif t == ')':
        currentTree = stack.pop()
    else:
        raise ValueError
    return tree

# Recursively evaluate the parse tree
def evaluate(self, tree, var_storage_statement):
    leftTree = tree.getLeftTree()
    rightTree = tree.getRightTree()
    op = tree.getKey()

    # evaluate base on the pmdas rule
    if leftTree is not None and rightTree is not None:

        leftValue = self.evaluate(leftTree, var_storage_statement)
        rightValue = self.evaluate(rightTree, var_storage_statement)

        # Check if either operand is a string (alphabet)
        if isinstance(leftValue, str) or isinstance(rightValue,
str):

            # If so, return None
            return None

        # Replace variable names with their values
        if isinstance(leftValue, str) and leftValue in
var_storage_statement:
            # print("HERE1")

```

```
        leftValue = var_storage_statement[leftValue]

        if isinstance(rightValue, str) and rightValue in
var_storage_statement:
            # print("HERE2")
            rightValue = var_storage_statement[rightValue]

    if op == '+':
        try:
            return leftValue + rightValue
        except TypeError:
            pass
    elif op == '-':
        try:
            return leftValue - rightValue
        except TypeError:
            pass
    elif op == '*':
        try:
            return leftValue * rightValue
        except TypeError:
            pass
    elif op == '/':
        try:
            return leftValue / rightValue
        except ZeroDivisionError:
            pass
        except TypeError:
            pass
    elif op == '**':
        return leftValue ** rightValue

    else:
        # If the current node is a variable, return its value from
the dictionary
        if tree.getKey() in var_storage_statement:
            return var_storage_statement[tree.getKey()]
        else:
```



```
return tree.getKey()
```

stack.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAA/2B/01

class Stack:
    def __init__(self):
        self.__list= []

    def isEmpty(self):
        return self.__list == []

    def size(self):
        return len(self.__list)

    def clear(self):
        self.__list.clear()

    def push(self, item):
        self.__list.append(item)

    def pop(self):
        if self.isEmpty():
            return None
        else:
            return self.__list.pop()

    def get(self):
        if self.isEmpty():
            return None
        else:
            return self.__list[-1]

    def __str__(self):
        output = '<'
```

```
for i in range( len(self.__list) ):
    item = self.__list[i]
    if i < len(self.__list)-1 :
        output += f'{str(item)}, '
    else:
        output += f'{str(item)}'
output += '>'
return output
```

tokenization.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

import re

class Tokenization:
    def tokenize(exp):
        # Operators to be considered as separate tokens
        operators = ['+', '-', '*', '/', '**']

        tokens = []
        current_token = ''

        is_unary_minus = True # Flag to track unary minus

        try:

            for char in exp:

                # Skip whitespace characters
                if char.isspace():
                    continue

                # alnum returns "true" if the char is a letter or a
                number

                if char.isalnum() or char == '.':
```

```
        current_token += char # char is appended into
current_token variable
        is_unary_minus = False # Reset the unary_minus flag

    elif char == '-' and is_unary_minus:
        # Handle unary minus
        current_token += char
        is_unary_minus = False

    elif char in operators:
        # Handle operators
        if current_token:
            tokens.append(current_token)
            current_token = ''

        if tokens and tokens[-1] == char:
            # Combine consecutive asterisks
            tokens[-1] = '**'
        else:
            tokens.append(char)

        is_unary_minus = True # Reset the unary_minus flag

    elif char == '(':
        # Handle opening parenthesis
        if current_token:
            tokens.append(current_token)
            current_token = ''

        tokens.append(char)
        is_unary_minus = True # Reset the unary_minus flag

    elif char == ')':
        # Handle closing parenthesis
        if current_token:
            tokens.append(current_token)
            current_token = ''
```

```
        tokens.append(char)

    else:
        # Raise an error for invalid characters
        raise ValueError(f"Invalid character: {char}")

    if current_token:
        tokens.append(current_token)

except ValueError as e:
    print(f"\nError during tokenization: {e}")

return tokens
```

utility.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

# Imports
import string
import os
import random

class Utility:

    #
    -----Inputs-----

    # Get the user input for integers
    def enterChoice(prompt):
        while True:
            try:
                choice = int(Utility.userInput(prompt))
                return choice
```

```
# only happens when ans given is not an integer
except ValueError:
    print('Invalid input. Please enter a valid integer.')

# Get the user input if not it just waits for the users reply
def userInput(prompt = None):
    if prompt == None:
        enter = input('Press enter key, to continue...')
        return enter
    else:
        return input(prompt)

# Get the user input (and ensure its not empty)
def userInputNotEmpty(prompt):
    while True:
        ans = Utility.userInput(prompt)

        if ans.strip(): # Check if the input is not empty after
stripping whitespace
            return ans
        else:
            print("Input cannot be empty. Please try again.")

# -----Commonly used (basic)
functions-----

# Read the content of the file
def readFile(filePath):
    with open(filePath, 'r') as file:
        return file.read()

# Write the output to a new file
def writeFile(filePath, content):
    with open(filePath, 'w') as file:
        return file.write(content)

# Ensure that the output file path is in the folder
def outputFilePathinFolder(folderName, fileName):
```

```
        return os.path.join(folderName, fileName)

def processAssignmentStatements(content, statement_storage):
    # Split the content into lines
    lines = content.split('\n')

    # Process each line (assignment statement)
    for line in lines:
        # Remove leading and trailing whitespaces
        line = line.strip()

        # Skip empty lines
        if not line:
            continue

        # Update the dictionary to store the assignment statements
        var, exp = line.split('=')
        statement_storage[var] = exp

#
-----Getters-----
-----

# Get and ensure that file path is valid and not empty
def getFile(filePathPrompt):
    while True:
        filePath = Utility.userInput(filePathPrompt)

        if Utility.__validateFilePath(filePath):
            return filePath
        else:
            print('Invalid input. Please enter a valid file path.')

def getOutputFile(message, content, variable_values):
    while True:
        # Get the output file path from the user
        output_file_path = Utility.userInput(f'{message}: ')
```

```
        # Call __validateVarName and return its output file path
        outputPath = Utility.__validateOutFile(output_file_path,
content, variable_values)
        if outputPath:
            return outputPath

#
-----Validate-----
-----

# validate the variable name
def validateVarName():
    while True:
        try:
            # Ask for the user input
            statement = Utility.userInput('Enter the assignment
statement you want to add/modify:\nFor example, a=(1+2)\n')

            # spilt the var and experssion
            var, exp = statement.split('=', 1)

            # Remove spaces from var and exp and check each
character
            var = var.strip()
            exp = exp.strip()

            if var.isalpha() and len(var) >= 1: # Check if var
contains only alphabets and has more than or equal to one letter
                # Check if exp is not empty
                if exp:

                    # Check for incomplete statements without any of
the following operators
                    if not any(op in exp for op in ['+', '-', '*',
('/', '**')):
                        print('Invalid statement. Please include at
least one of the operators: +, -, *, /, **')
                        continue
```

```
        # Check for incomplete statements
        elif exp.endswith(('+', '-', '*', '/', '**')):
            print('Invalid incomplete statement. Please
enter a complete assignment.')
            continue

        invalid_char = False
        for i in range(len(exp)):
            char = exp[i]
            prev_char = exp[i - 1] if i > 0 else None
            next_char = exp[i + 1] if i < len(exp) - 1
else None

            if not (char.isalnum() or
                    (char == '.' and
(prev_char.isnumeric() if prev_char else True) and
(next_char.isnumeric() if next_char else True))
                    or char in ['+', '-', '*', '/',
'**', '(', ')', ' ']):
                invalid_char = True
                break

        if invalid_char:
            print('Invalid characters in the expression.
Please use only alphanumeric characters, decimal point, and valid
operators.')
            continue

        # Check for valid brackets
        elif not Utility.checkBrackets(exp):
            print("Invalid bracket usage. Please check
your brackets.")
            continue

        # Ensure that the expression contains at least
one pair of brackets
        elif '(' not in exp or ')' not in exp:
```



```
                print("Invalid expression. Please include at
least one pair of brackets.")
                continue

            return var, exp

        else:
            print('Invalid incomplete statement. Please
enter a complete assignment.')
        else:
            print('Please key in a statement with 1 variable at
the start (e.g., a=(1+2))')

    except ValueError:
        print('Invalid input. Please enter a valid statement.')

# Validate the file path
def __validateFilePath(filePath):
    try:
        with open(filePath, 'r'):
            pass # Just open and immediately close the file to
check if it exists
        return True
    except FileNotFoundError:
        return False
    except OSError:
        return False

# write to output file
def __validateOutFile(output_file_path, content, variable_values):
    while True:

        # Check if the output file already exists
        if os.path.exists(output_file_path):
            print("Output file already exists. Please choose a
different name.")
            return
```

```

        # Group statements by their evaluated values
        grouped_statements = {None: []} # Initialize with None as a
key

        for var, exp in content:
            correct_exp = exp.replace(" ", "")
            try:
                result = eval(exp, variable_values)
            except NameError:
                result = None

            # checks whether the result is evaluated and is inside
grouped statements
            if result not in grouped_statements:
                grouped_statements[result] = [] # if the result not
inside then create a new list that will store statements that evaluate
the same value

                grouped_statements[result].append((var, correct_exp)) #
appends a tuple to the list associated with the result key in the
grouped statements whcih contains the var and exp

        # Sort the dictionary by the keys (values) in descending
order
        sorted_statements = sorted(grouped_statements.items(),
key=lambda x: (float('-inf') if x[0] is None else x[0], x[0]),
reverse=True)

        # '-inf' ensure that 'None' is considered the lowest value
during sorting

        # the second x[0] is the original evaluated result which
ensures that if there are statements with the same evaluated result, it
will be further sorted based on thier original evaluated result

        # Write to the output file
        try:
            with open(output_file_path, 'w') as output_file:
                for value, statements in sorted_statements:
                    output_file.write(f'\n*** Statements with

```

```
value=> {value}\n')
        for var, exp in statements:
            output_file.write(f'{var} = {exp}\n')
        break # Exit the loop if writing is successful
except Exception as e:
    print(f"Error writing to the output file: {e}")

return output_file_path

#
-----Others-----
-----

# Check the brackets
def checkBrackets(expression):
    stack = []
    left_bracket = '('
    right_bracket = ')'

    for char in expression:
        if char == left_bracket:
            stack.append(char)
        elif char == right_bracket:
            if not stack or stack.pop() != left_bracket:
                return False

    return not stack # Stack should be empty if brackets are
balanced

# Generate Random Strings of Data
def generateRandomData(size, str_length):
    characters = string.ascii_letters + string.digits
    data = [''.join(random.choice(characters) for _ in
range(str_length)) for _ in range(size)]
    random.shuffle(data) # Shuffle the list to ensure it's not
already sorted
    return data
```

drawTree.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

## OPTION 6
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tabulate import tabulate

import re

class DrawTree:

    def __init__(self, statement_storage):
        self.statement_storage = statement_storage # get the current
stored statements
        self.variable_values = {}
        self.sorted_statements = None

        # ensure that there are expressions stored in the statement storage
    def build_tree_data(self, statement_storage):
        tree_data = []
        for var, exp in statement_storage.items():
            matches = re.findall(r'\b\w+\b', exp) # find all the
statements that matches the current expression
            for match in matches:
                if match in statement_storage:
                    tree_data.append((var, match)) # append the
expression into the tree_data list
        return tree_data

        # draws the graph
    def draw_tree(self, tree_data):
        g = nx.Graph()
        g.add_edges_from(tree_data)
```

```

        pos = nx.spring_layout(g)
        nx.draw(g, pos, with_labels=True, font_size=8,
font_weight="bold", node_size=700, node_color="skyblue")
        plt.show()

    # building the dependency matrix
    def build_dependency_matrix(self, statement_storage):
        variables = list(statement_storage.keys())
        matrix_size = len(variables)

        # Initialize the dependency matrix with zeros
        dependency_matrix = np.zeros((matrix_size, matrix_size),
dtype=int)

        for i, var in enumerate(variables):
            exp = statement_storage[var]
            matches = re.findall(r'\b\w+\b', exp) # find all the
variables in the expressions

            for match in matches:
                if match in variables: # check that matched variable
exists
                    j = variables.index(match) # get the index 'j'
                    dependency_matrix[i, j] = 1 # updates corresponding
entry in the dependency matrix

        return variables, dependency_matrix

    # displaying the dependency matrix
    def display_dependency_matrix(self, variables, dependency_matrix):
        df = pd.DataFrame(dependency_matrix, index=variables,
columns=variables) # store it in a dataframe
        table = tabulate(df, headers='keys', tablefmt='grid',
showindex=True, numalign="center", stralign='center') # keys specifies
the column names be used as header
        print("\nDependency Matrix:")
        print(table) # prints the formatted table

```

```
# call the results
def visualize_dependency_info(self):
    variables, dependency_matrix =
self.build_dependency_matrix(self.statement_storage)
    self.display_dependency_matrix(variables, dependency_matrix)
```

turtleGraph.py

```
# # Name: Teng Tze Hui & Ng Jace Xin
# # Student ID: 2214209 & 2214593
# # Class: DAAA/2B/01

import turtle
import sympy as sp
import numpy as np
import re

class EquationGraph:
    def __init__(self):
        self.equations = []

    def is_valid_equation(self, equation_str):
        # Check if 'x' and 'y' are present in the equation
        if 'x' in equation_str and 'y' in equation_str:
            return True
        return False

    def input_equations(self):
        while True:
            try:
                num_equations = int(input("Enter the number of equations
(must be a positive integer): "))
                if num_equations > 0:
                    break # Break out of the loop if a valid input is
provided
            else:
                print("Please enter a positive integer greater than
```

```

zero.")
        except ValueError:
            print("Invalid input. Please enter a positive integer.")

    for i in range(num_equations):
        while True:
            equation_str = input(f"Enter equation {i + 1} in a
suitable format (eg. y = 2 * x + 1): ")

            # Validate the input format
            if self.is_valid_equation(equation_str):
                break # Break out of the loop if a valid input is
provided
            else:
                print("Invalid input. Please enter 'x' and 'y' in
the equation.")

            if '=' in equation_str:
                _, equation_str = equation_str.split('=')

            self.equations.append(sp.simplify(equation_str))

    def customize_graph(self):
        color = input("Enter the color for the graph (default is blue):
") or "blue"
        linestyle = input("Enter the linestyle for the graph ('-' or
'-.') (default is solid line): ") or "-"
        return color, linestyle

    def draw_graph(self, color="blue", linestyle="-"):
        turtle.speed("fastest")
        turtle.title("Equation Solver Game")
        turtle.bgcolor("white")
        turtle.setup(width=800, height=600)

        # Draw x-axis
        turtle.penup()

```

```
turtle.goto(-400, 0)
turtle.pendown()
turtle.forward(800)

# Draw y-axis
turtle.penup()
turtle.goto(0, -300)
turtle.pendown()
turtle.setheading(90)
turtle.forward(600)

for equation in self.equations:
    self.draw_single_equation(equation, color, linestyle)

# Implement interactive features
turtle.onscreenclick(self.on_click)
turtle.done()

def draw_single_equation(self, equation, color, linestyle):
    turtle.speed("fastest")

    x_vals = np.linspace(-10, 10, 400)
    y_vals = np.array([sp.N(equation.subs('x', val)) for val in
x_vals])

    turtle.penup()
    turtle.color(color)
    turtle.goto(x_vals[0] * 40, y_vals[0] * 40)

    turtle.pendown()
    turtle.pencolor(color)
    turtle.pensize(2)
    turtle.setheading(90)
    turtle.pendown()

    # Set linestyle
    if linestyle == "-.":
        for i in range(1, len(x_vals), 3):
```



```
        turtle.speed("fastest")
        turtle.penup()
        turtle.goto(x_vals[i] * 40, y_vals[i] * 40)
        turtle.pendown()
        turtle.goto(x_vals[i + 2] * 40, y_vals[i + 2] * 40)
    else:
        for i in range(1, len(x_vals)):
            turtle.speed("fastest")
            turtle.penup()
            turtle.goto(x_vals[i - 1] * 40, y_vals[i - 1] * 40)
            turtle.pendown()
            turtle.goto(x_vals[i] * 40, y_vals[i] * 40)

    # Reset linestyle
    turtle.pendown()
    turtle.pencolor(color)
    turtle.pensize(2)

    def on_click(self, x, y):
        print(f"Clicked at ({x}, {y})")
```

graph.py

```
# Name: Teng Tze Hui & Ng Jace Xin
# Student ID: 2214209 & 2214593
# Class: DAAA/2B/01

# Import
import networkx as nx
import matplotlib.pyplot as plt

class GraphUtility:

    # Create a simple networkx graph
    def createGraph(edges):

        # Create the graph
        g = nx.Graph()
```

```
        # Iterate through the edges and add them to the graph with
weights
        for edge in edges:

            try:
                # Add edge to the graph with a weight (converted to
float)
                g.add_edge(edge[0], edge[1], weight=float(edge[2]))

            except IndexError:
                print('Error: Tuple index out of range. Returning to the
main menu...')
                return None

            except ValueError:
                print('Error: Could not convert string to float.
Returning to the main menu...')
                return None

        return g

    # Find the shortest distance path
    def findShortestDistance(g, start_node, end_node):

        try:
            # Use NetworkX functions to find the shortest path and its
length
            shortest_path = nx.shortest_path(g, start_node, end_node,
weight='weight')
            dist = nx.shortest_path_length(g, start_node, end_node,
weight='weight')
            print('Shortest path is:', shortest_path, '\nWith total
distance of', dist, 'units')

        except nx.NodeNotFound as e:
            print(f'Error: {e}. Returning to the main menu...')
```

```
def visualizeGraph(g, edges, choice):  
  
    # Visualize the graph using spring layout  
    if choice == 1:  
        pos = nx.spring_layout(g)  
  
    # Visualize the minimum spanning tree of the graph using spring  
    layout  
    elif choice == 2:  
        mst = nx.minimum_spanning_tree(g)  
        pos = nx.spring_layout(mst)  
  
    # Visualize the directed graph using spring layout  
    elif choice == 3:  
        # Create the graph  
        g = nx.DiGraph()  
  
        for edge in edges:  
            g.add_edge(edge[0], edge[1])  
        pos = nx.spring_layout(g)  
  
    # Plot the selected graph  
    nx.draw(g, pos=pos, with_labels=True, node_size=500 if choice ==  
3 else None)  
    plt.show()
```

sort.py

```
# Name: Teng Tze Hui & Ng Jace Xin  
# Student ID: 2214209 & 2214593  
# Class: DAA/2B/01  
  
class Sort:  
  
    # Initialize the steps attribute  
    def __init__(self):  
        self.steps = 0
```

```
# Bubble Sort Function (implementation)
def bubbleSort(self, l):
    self.steps = 0 # Reset steps for each sorting operation
    isSorted = False

    while not isSorted:
        isSorted = True
        for i in range(len(l)-1): # represents one pass of swapping.
            self.steps += 1 # Increment steps for each comparison
            if l[i]>l[i+1]:
                l[i],l[i+1] = l[i+1],l[i]
                isSorted = False

# Merge Sort Function (implementation)
def mergeSort(self, l):
    self.steps = 0 # Reset steps for each sorting operation
    if len(l) > 1:
        mid = int (len(l)/2)

        # Here is where we split into two halves.
        leftHalf = l[:mid]
        rightHalf = l[mid:]

        # Recursive call
        Sort.mergeSort(self, leftHalf)
        Sort.mergeSort(self, rightHalf)

        leftIndex,rightIndex,mergeIndex = 0,0,0

        # take cares of the merging
        mergeList = l
        while leftIndex < len(leftHalf) and rightIndex <
len(rightHalf):
            self.steps += 1 # Increment steps for each comparison

            if leftHalf[leftIndex] < rightHalf[rightIndex]:
                mergeList[mergeIndex] = leftHalf[leftIndex]
```

```
        leftIndex+=1
    else:
        mergeList[mergeIndex] = rightHalf[rightIndex]
        rightIndex+=1
        mergeIndex+=1

    # Handle those items still left in the left Half
    while leftIndex < len(leftHalf):
        mergeList[mergeIndex] = leftHalf[leftIndex]
        leftIndex+=1
        mergeIndex+=1

    # Handle those items still left in the right Half
    while rightIndex < len(rightHalf):
        mergeList[mergeIndex] = rightHalf[rightIndex]
        rightIndex+=1
        mergeIndex+=1
```